

Buenas prácticas de API REST

Primero vamos a cambiar el path base de / blueprints a la ruta /api/v1/blueprints.

Objetivo

- Permitir evolución futura sin romper clientes existentes.
- Facilitar mantenimiento y control de versiones.
- Adoptar estándar REST moderno.

Prueba HTTP

Mensaje 201

The screenshot shows a REST client interface with the following sections:

- Responses**: A header for the response section.
- Curl**: A text area containing a curl command for a POST request to `http://localhost:8080/api/v1/blueprints` with headers `accept: */*` and `Content-Type: application/json`. The body is a JSON object: `{ "author": "Felipe", "name": "plano2", "points": [ { "x": 4, "y": 5 } ] }`.
- Request URL**: A text field containing `http://localhost:8080/api/v1/blueprints`.
- Server response**: A section showing the response details.
 - Code**: 201
 - Details**: A table with the following content:

Code	Details
201	Response headers
Undocumented	connection: keep-alive content-length: 0 date: Thu, 19 Feb 2026 22:58:16 GMT keep-alive: timeout=60
- Responses**: A footer for the response section.

El mensaje 201 hace referencia a un recurso creado correctamente

Mensaje 200

```
curl -X 'GET' \
  'http://localhost:8080/api/v1/blueprints' \
  -H 'accept: */*' \
```

Request URL

http://localhost:8080/api/v1/blueprints

Server response

Code	Details
200	Response body <pre>[{ "author": "Felipe", "name": "plano2", "points": [{ "x": 4, "y": 5 }] }, { "author": "oscar", "name": "plano1", "points": [{ "x": 1, "y": 2 }, { "x": 3, "y": 4 }] }]</pre> Download

Hace referencia a una consulta correcta

Mensaje 202

Curl

```
curl -X 'PUT' \
  'http://localhost:8080/api/v1/blueprints/Felipe/plano2/points' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "x": 1,
    "y": 1
  }'
```

Request URL

http://localhost:8080/api/v1/blueprints/Felipe/plano2/points

Server response

Code	Details
202 <i>Undocumented</i>	Response headers <pre>connection: keep-alive content-length: 0 date: Thu, 19 Feb 2026 23:03:00 GMT keep-alive: timeout=60</pre>

Responses

Code	Description	Links
200	OK	No links

Modificación aceptada

Mensaje 404

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:8080/api/v1/blueprints/felipe/plano2/points' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "x": 4,
    "y": 4
  }'
```

Request URL

```
http://localhost:8080/api/v1/blueprints/felipe/plano2/points
```

Server response

Code	Details
404 <i>undocumented</i>	Error: response status is 404

Response body

```
{
  "error": "Blueprint not found"
}
```

 **Download**

Response headers

```
connection: keep-alive
content-type: application/json
date: Thu, 19 Feb 2026 23:01:12 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

Code	Description	Link
------	-------------	------

Recurso no encontrado

Mensaje 400

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/v1/blueprints' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "author": "Felipe",
    "name": "plano2",
    "points": [
      {
        "x": 4,
        "y": 0
      }
    ]
  }'
```

Request URL

http://localhost:8080/api/v1/blueprints

Server response

Code	Details
400 <i>undocumented</i>	Error: response status is 400

Response body

```
{
  "timestamp": "2026-02-19T23:04:21.367+00:00",
  "status": 400,
  "error": "Bad Request",
  "path": "/api/v1/blueprints"
}
```

Datos erróneos

Crear respuesta uniforme ApiResponse<T>

Se implementó una estructura uniforme para todas las respuestas

```
package edu.eci.arsw.blueprints.dto;

public record ApiResponse<T>(
    int code,
    String message,
    T data
) {}
```

Beneficios

- Respuestas consistentes.
- Manejo uniforme de errores.
- Mayor claridad semántica.

Luego Se ajustaron los códigos de respuesta según la operación

```
// GET /blueprints
@GetMapping
public ResponseEntity<ApiResponse<Set<Blueprint>>> getAll() {
    Set<Blueprint> data = services.getAllBlueprints();
    return ResponseEntity.ok(
        new ApiResponse<>(200, "execute ok", data));
}

// GET /blueprints/{author}
@GetMapping("/{author}")
public ResponseEntity<?> byAuthor(@PathVariable String author) {
    try {
        Set<Blueprint> data = services.getBlueprintsByAuthor(author);
        return ResponseEntity.ok(
            new ApiResponse<>(200, "execute ok", data));
    } catch (BlueprintNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiResponse<>(404, e.getMessage(), null));
    }
}
```

```

// GET /blueprints/{author}/{bpname}
@GetMapping("/{author}/{bpname}")
public ResponseEntity<?> byAuthorAndName(@PathVariable String author,
    @PathVariable String bpname) {
    try {
        Blueprint data = services.getBlueprint(author, bpname);
        return ResponseEntity.ok(
            new ApiResponse<>(200, "execute ok", data));
    } catch (BlueprintNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiResponse<>(404, e.getMessage(), null));
    }
}

// POST /blueprints
@PostMapping
public ResponseEntity<ApiResponse<?>> add(@Valid @RequestBody NewBlueprint req) {
    try {
        Blueprint bp = new Blueprint(req.author(), req.name(), req.description());
        services.addNewBlueprint(bp);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(new ApiResponse<>(201, "Blueprint created", null));
    } catch (BlueprintPersistenceException e) {
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(new ApiResponse<>(403, e.getMessage(), null));
    }
}

```

```

// PUT /blueprints/{author}/{bpname}/points
@PutMapping("/{author}/{bpname}/points")
public ResponseEntity<?> addPoint(@PathVariable String author, @PathVariable
    @RequestBody Point p) {
    try {
        services.addPoint(author, bpname, p.x(), p.y());
        return ResponseEntity.status(HttpStatus.ACCEPTED)
            .body(new ApiResponse<>(202, "Point added", null));
    } catch (BlueprintNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ApiResponse<>(404, e.getMessage(), null));
    }
}

public record NewBlueprintRequest(
    @NotBlank String author,
    @NotBlank String name,
    @Valid java.util.List<Point> points) {
}

```

Se implementó un `@RestControllerAdvice` para capturar errores de validación:

```

package edu.eci.arsw.blueprints.exceptions;

import edu.eci.arsw.blueprints.dto.ApiResponse;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ApiResponse<?> handleValidation(MethodArgumentNotValidException e) {
        return new ApiResponse<>(400, "Invalid request data", null);
    }
}

```

Esto evita respuestas HTML por defecto y mantiene el estándar JSON.